CG2271 Real Time Operating Systems

2025/26 Semester 2

Final Report

B02_03	
Name	Matric Number
Ariff Muhammed Ahsan Husain	[redacted]
T. Kandasami	[redacted]
Kevin Loke Wei Yi	[redacted]
Karthikeyan Vetrivel	[redacted]

1. Problem Statement

Students studying at their desks are frequently unaware of their environment, such as noise, rising temperature, poor lighting, and ambient noise. These factors silently erode concentration and productivity. So let us introduce Baymax, an embedded real-time system that continuously monitors study conditions by processing sensor data and proactively advising students on proper action to maintain an optimal study environment.

2. User Story

As a university student studying at my desk, I want the system to immediately alert me via a buzzer or RGB LED when my room becomes too hot, too dim or too noisy so that I can take corrective action promptly by referencing an AI-generated advice message on Telegram explaining what condition is unideal and what I should do.

3. Feature List

Baymax is a real-time embedded study environment monitor built on a dual-microcontroller architecture comprising the MCXC444 and ESP32S2. The system continuously tracks ambient light, sound, temperature, and humidity, alerting the user via a buzzer and an RGB LED when conditions degrade. It is also equipped with an OLED interface showing a timer of their study session, along with other functionalities.

1. Real-time monitoring of light, sound, temperature and humidity throughout the study session
2. Study session management via tap sensor: Double tap to start or stop & Single tap to pause or resume
3. RGB LED providing visual feedback of environment quality, smoothly blending from green through orange to red based on an environment score
4. Passive buzzer alert when environment conditions are poor, with 60-second cooldown to avoid being disruptive
5. OLED display showing live session timer, current temperature and humidity, real-time environment emoji, session summary on stop, and scrolling AI-generated report
6. Exponential Moving Average (EMA) filtering on all sensor readings to smooth out noise and prevent false triggers
7. AI-generated personalised study environment report delivered at session end via the Gemini API
8. Instant Telegram notification delivering the AI report to the chat in real time

4. Sensors and Actuators

4.1 Sensors

Sensor	Description of use	Libraries used
DHT11	The DHT11 temperature and humidity sensor is used to record ambient temperature and humidity.	DHT Sensor Library by Adafruit
Big Sound Sensor	Connected to the MCXC444 via ADC. Continuously samples ambient noise, performs startup calibration to determine a baseline, and detects large deviations from that baseline. When a spike is detected, sound_triggered is set to 1 and transmitted to the ESP32 over UART.	No external library used
Switch Button	Connected to the MCXC444 via PTD2 as a falling-edge interrupt. A double tap starts or resumes the session, and a single tap pauses it. Interrupt-driven because tap events are instantaneous and must feel responsive.	No external library used
Photoresistor (LDR)	Connected to the MCXC444 via ADC. Continuously monitors ambient light level. Since it shares ADC0 with the sound sensor, a mutex is used for synchronisation. Higher ADC values indicate dimmer conditions.	No external library used

4.2 Actuators

Actuator	Description of Use	Libraries Used
OLED	Connected to the ESP32 via I2C (SDA=GPIO6, SCL=GPIO7). Displays the study session timer, pause state with a pause symbol, session summary on stop, and a scrolling Gemini AI response. Also shows a real-time facial emoji in the top-right corner reflecting the current environment condition.	Adafruit SSD1306, Adafruit GFX.
LED	Connected to the MCXC444. Provides a continuous visual indication of study environment quality using PWM-driven RGB LED channels. The LED smoothly blends from green when conditions are good, through orange when conditions are moderate, to red when conditions are poor. The colour is driven by a blend value computed by vEnvTask based on sound trigger count, light level, and temperature.	No external library used
Passive Buzzer	Connected to the MCXC444. Beeps periodically when the environment condition is in the red state, audibly alerting the user to take corrective action.	No external library used

5. Hardware Architecture

5.1 MCXC444 Connections

Component	Pin	Mode
Switch Button	PTD2	Digital input, falling-edge interrupt
Sound Sensor	PTB0 (ADC0)	Analogue input, polled
Photoresistor (LDR)	PTB2 (ADC0_SE12)	Analogue input, Interrupt driven ADC
RGB LED	PTA5 (RED) PTE30 (GREEN) Disabled (BLUE)	Digital output
Passive Buzzer	PTA1	Digital output
UART2 TX (to ESP32)	PTE22	UART transmit
UART2 RX (from ESP32)	PTE23	UART receive

5.2 ESP32S2 Connections

Component	Pin	Mode
DHT11	GPIO20	Digital input, single-wire protocol
OLED SDA	GPIO6	I2C data
OLED SCL	GPIO7	I2C clock
UART TX (to MCXC444)	GPIO16	UART transmit
UART RX (from MCXC444)	GPIO17	UART receive

5.3 Polling & Interrupt Assignment Justification

Each sensor's read method was chosen based on the nature of its signal and the responsiveness requirement of the application.

The switch button (PTD2) is interrupt-driven via a falling-edge interrupt on PORTC_PORTD_IRQHandler. A tap lasts only a few milliseconds and occurs at an unpredictable time. Polling it periodically would risk missing a short press entirely. The interrupt fires the exact moment the button is pressed, guaranteeing zero missed events. The ISR does only gives gTapSemaphore using xSemaphoreGiveFromISR() and returns immediately, keeping interrupt latency minimal.

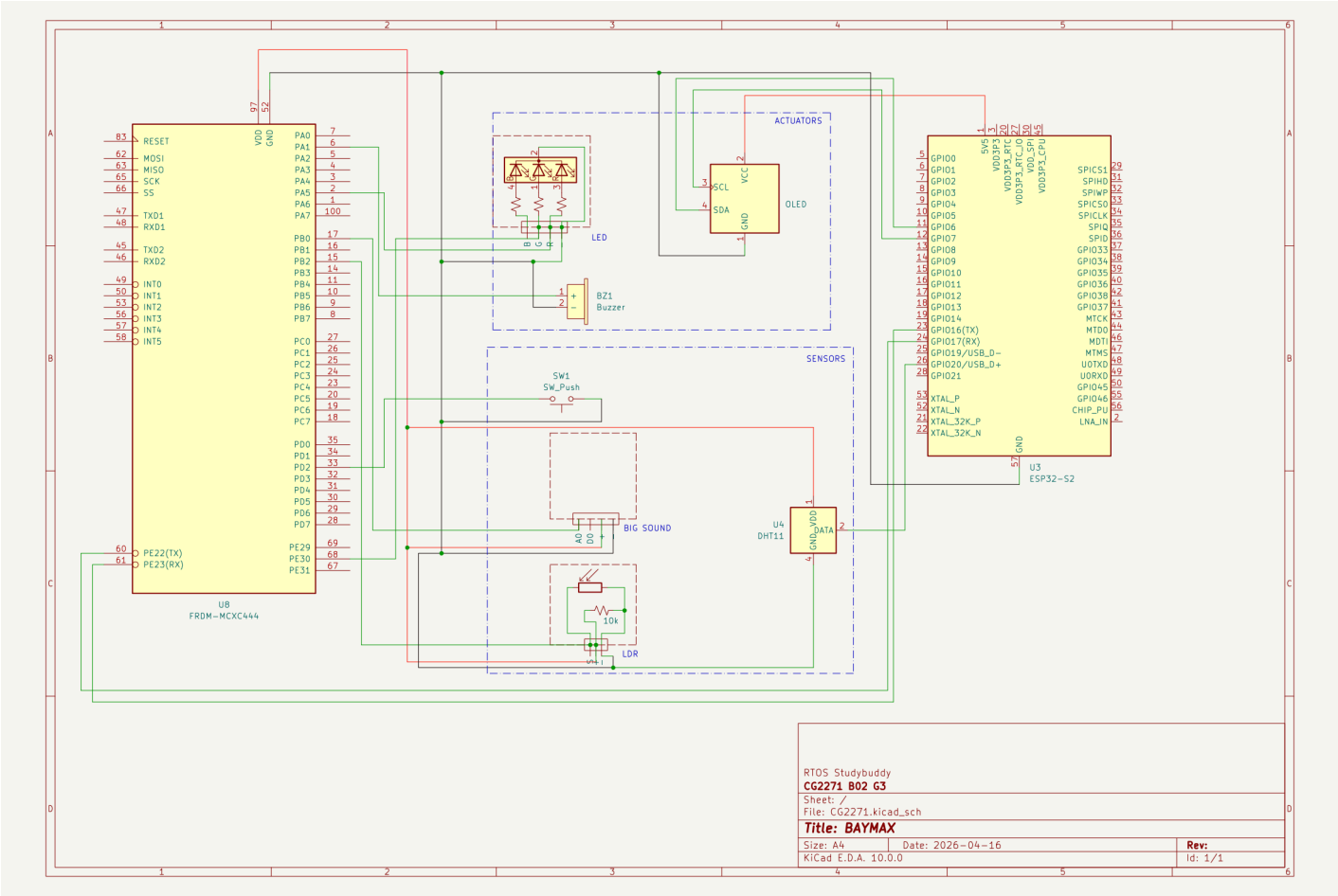
The light sensor (PTB2) uses interrupt-driven ADC conversion. LIGHT_SENSOR_Task wakes every 500ms and triggers a conversion by writing to ADC0->SC1[0]. Rather than busy-waiting for the ADC to finish, the task blocks on xLightQueue. When the hardware completes the conversion, ADC0_IRQHandler fires, reads the result register, and posts the value to the queue using xQueueOverwriteFromISR(), waking the task. The 500ms period is appropriate because light levels change gradually and do not require sub-second resolution. The interrupt-driven completion avoids wasting CPU cycles busy-waiting for the ADC result.

The sound sensor (PTB0) uses continuous polling within its task. Sound is a continuous analogue signal with no single discrete event to interrupt on. vSoundTask takes 400 rapid ADC samples per cycle to compute the peak, trough, and deviation count from a calibrated baseline. This burst sampling approach is necessary to reliably detect sustained noise spikes rather than isolated ADC fluctuations. A rolling 30-second trigger counter tracks how many noise events occurred.

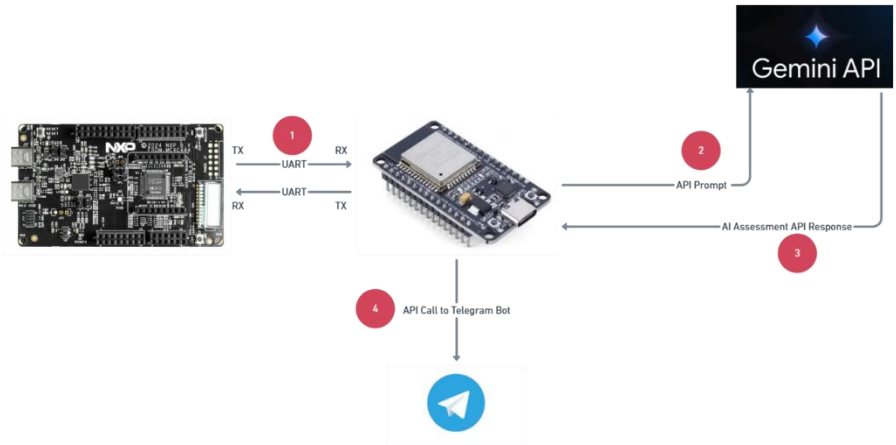
The DHT11 temperature and humidity sensor (GPIO20) is polled every 1000ms by vDHTTask. Temperature and humidity change very slowly, meaning changes occur over tens of seconds to minutes rather than milliseconds. Polling every second provides more than sufficient resolution. Between readings the task calls vTaskDelay(pdMS_TO_TICKS(DHT_POLL_MS)) which yields the CPU entirely, allowing other tasks to run freely during the wait

5.4 Circuit Diagram

The circuit diagram below shows the connections made between the components and the devices.



6. Software Architecture



The system is divided across two microcontrollers with clearly defined responsibilities. The MCXC444 runs eight FreeRTOS tasks handling all time-critical sensing and actuation: it monitors ambient light via the photoresistor, detects noise spikes via the sound sensor, tracks study session state via the switch button interrupt, classifies the overall environment condition, drives the RGB LED and passive buzzer, and communicates with the ESP32 over UART2. ESP32 complements this by reading temperature and humidity from the DHT11 sensor, aggregating all incoming sensor data from the MCXC444, and managing cloud connectivity. At the end of each study session, the full dataset, including average temperature, humidity, light level, sound level, and

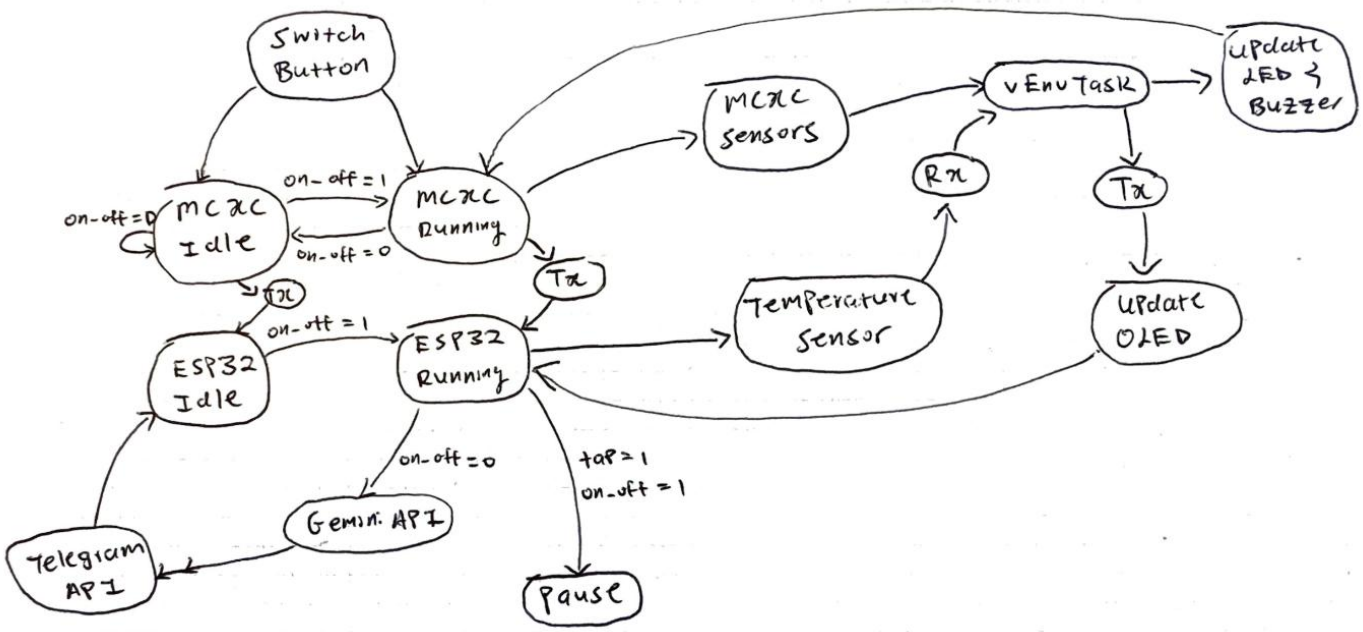
environment condition, is sent to the Gemini API to generate a personalised study environment report. This report is delivered to the user via a Telegram group chat and simultaneously scrolled across the OLED display.

6.1 State Management

The system is event-driven around a study session lifecycle managed on the MCXC444 and mirrored on the ESP32. The session has three main states: Idle, Running and Paused. The MCXC444 detects state transitions via the switch button interrupt and encodes the current state in two fields, on_off and paused, transmitted every packet over UART. On the ESP32 side, vUartRxTask detects transitions by comparing the current on_off and paused values against their previous values (sPrevOnOff, sPrevPaused).

START is detected when on_off transitions from 0 to 1, PAUSED when paused transitions from 0 to 1 while on_off remains 1, RESUME when paused transitions from 1 to 0, and a stop when on_off transitions from 1 to 0. At the end of a session, the Gemini binary semaphore gGeminiSemaphore is given, unblocking vGeminiTask to generate the session report.

6.2 Finite State Machine Diagram



6.3 Task Structure

6.3.1 Tasks on MCXC444 Board

Task	Priority	Stack Size (4 Bytes)	Responsibility
vTapTask	4 (highest)	90	Blocks on gTapSemaphore; detects single vs double tap; toggles on_off or paused in gSensorData; notifies vTxTask immediately
vBuzzerTask	3	90	Reads gBuzzerRequest; emits a short beep (150ms) on tap events or a long beep (800ms) when environment is poor, with a 60s cooldown between environment alerts
vEnvTask	3	90	Blocks on gTempReadySem; computes exponential moving averages for light, sound, temperature and humidity; calculates environment condition score; sets g_color_blend for LED; triggers buzzer on poor conditions; notifies vTxTask
vLightTask	2	90	Triggers interrupt-driven ADC conversion on PTB1 every 500ms; writes raw ADC value to gSensorData under gSensorMutex
vSoundTask	2	90	Performs ADC calibration at startup to establish baseline; continuously samples PTB0 for peak noise level; detects spikes above TRIGGER_DELTA; counts triggers per 30s window; writes to gSensorData
vLEDTask	2	90	Reads g_color_blend and led_state; drives RGB LED via PWM using TPM channels; smoothly transitions between green and red by incrementally adjusting duty cycle every 20ms
vTxTask	2	90	Waits for task notification from vEnvTask; reads averaged sensor data under gSensorMutex; builds 17-byte packet and sends to ESP32 via UART2
vRxTask	1 (Lowest)	90	Processes incoming bytes from xRxQueue (filled by ISR); parses LED command packets and temperature packets; updates led_state and gSensorData accordingly

6.3.2 Tasks on ESP32S2 Board

Task	Priority	Stack Size (Bytes)	Responsibility
vUartRxTask	4 (Highest)	2048	Receives and parses sensor packets from MCXC444 over UART1; detects session state transitions; gives Gemini semaphore on session end
vTelegramTask	3	6144	Blocks on gTelegramQueue; sends Gemini session report to Telegram group when a message is dequeued
vDHTTask	2	2048	Polls DHT11 every 1000ms; writes temperature and humidity to gSensorData; sends readings to MCXC444
vGeminiTask	2	6144	Blocks on gGeminiSemaphore; reads session data from gSensorData; constructs prompt and calls Gemini API; enqueues response

vOLEDTask	1 (Lowest)	4096	Reads session state from gSensorData every 100ms; renders running timer, pause screen, session summary, and scrolling Gemini response on the SSD1306 OLED display

6.4 Priority Assignment & Justification

Priority assignment works as such: 4 is the highest priority while 1 is the lowest. vUartRxTask is highest because UART data must be read immediately or bytes are lost from the hardware buffer. vTelegramTask is next as network sends are time-sensitive. vDHTTask and vGeminiTask share priority 2 as both are periodic and tolerate slight delays. vOLEDTask is lowest since display updates are purely cosmetic and can be pre-empted freely.

6.5 Coordination & Synchronisation

6.5.1 ESP32S2 Coordination & Synchronisation

Three synchronisation primitives are used. gGeminiSemaphore is a binary semaphore used to signal session end from vUartRxTask to vGeminiTask, the Gemini task blocks indefinitely on this semaphore and only unblocks when a stop event is detected. gTelegramQueue is a FreeRTOS queue of depth 1 holding a 512-byte Gemini response buffer, vGeminiTask writes to it after a successful API response, and vTelegramTask blocks on it waiting to send. gGeminiMutex protects the shared gGeminiOLEDMsg buffer and gGeminiOLEDReady flag, which are written by vGeminiTask and read by vOLEDTask to display the scrolling response on the OLED.

6.5.2 MCXC444 Coordination & Synchronisation

On the MCXC444, three synchronisation primitives are also used. gSensorMutex is a mutex protecting gSensorData and gAverageSensorData, which are shared across all sensor tasks, vEnvTask, and vTxTask. gADCMutex serialises access to the shared ADC0 hardware between LIGHT_SENSOR_Task and vSoundTask, preventing concurrent ADC conversions from corrupting each other's results. gTempReadySem is a binary semaphore given by vRXTask each time a valid temperature packet is received from the ESP32, which unblocks vEnvTask to recompute the environment condition with the latest thermal data. vTxTask uses a direct task notification from vEnvTask rather than a queue, allowing zero-copy, low-latency signalling that a new packet should be transmitted immediately.

6.6 Resource Protection

Three mutexes protect shared resources. gSensorMutex protects gSensorData, the shared struct written by both vDHTTask and vUartRxTask and read by vGeminiTask and vOLEDTask, all accesses are guarded with a short 10–100ms timeout to avoid priority inversion. gNetworkMutex serialises all WiFi operations, preventing vGeminiTask and vTelegramTask from making simultaneous HTTPS calls that would corrupt the underlying WiFi driver state. gGeminiMutex protects the OLED message buffer written by vGeminiTask and read by vOLEDTask. All mutexes are created before any tasks are spawned in setup(), ensuring no task can attempt to take an uninitialised handle.

7. Communication Protocol

The MCXC444 and ESP32 communicate bidirectionally over a single UART channel. Data flows in both directions simultaneously, the MCXC444 continuously sends sensor data packets to the ESP32, while

the ESP32 sends temperature readings and LED commands back to the MCXC444. All packets use a common framing structure with start markers, an XOR checksum, and an end marker to ensure integrity.

7.1 UART Configuration

A single UART channel handles bidirectional communication between the MCXC444 and the ESP32, configured at 115200 baud, 8 data bits, no parity, 1 stop bit. On the MCXC444 side, UART2 is used with TX on PTE22 and RX on PTE23. On the ESP32 side, UART1 is used with TX on GPIO16 and RX on GPIO17. The ESP32 driver is installed with a 256-byte RX hardware buffer, while the MCXC444 uses an interrupt-driven receive queue. The MCXC444 sends sensor data packets to the ESP32, and the ESP32 sends two types of packets back: an LED command packet and a temperature/humidity packet.

7.2 Packet description

Byte	Field	Description
0	START1	0xAA
1	START2	0x55
2	tap_event	1 = tap detected
3	on_off	1 = session active
4	paused	1 = session paused
5-6	light	16-bit ADC value (H byte first)
7-8	sound	16-bit ADC value (H byte first)
9	sound_triggered	1 = noise spike detected
10	env_condition	Environment state enum (GOOD/MODERATE/POOR/etc.)
11	temp	Temperature integer part
12	temp_frac	Temperature fractional part (tenths)
13	hum	Humidity integer part
14	hum_frac	Humidity fractional part (tenths)
15	checksum	XOR of bytes 2–14
16	END	0xBB

ESP32 to MCXC444 LED command (5 bytes): START1 (0xAA), START2 (0x55), cmd (0x01=ON / 0x00=OFF), checksum (0xAA XOR 0x55 XOR cmd), END (0xBB).

ESP32 to MCXC444 Temperature packet (8 bytes): START1 (0xAA), START2 (0x56), temp_int, temp_frac, hum_int, hum_frac, checksum (XOR of all data bytes), END (0xBB). The alternate START2 byte (0x56 vs 0x55) disambiguates this packet type from the LED command on the MCXC444 receiver.

Packet integrity is verified on both sides using XOR checksums and start/end marker validation. Invalid packets are silently dropped.

8. Online Functions Used

8.1 Gemini API (Google AI)

At the end of every study session, the ESP32 constructs a structured prompt containing the aggregated session data, including study duration, average temperature, average humidity, average light level, average sound level, sound spike count, and overall environment condition, and sends it via HTTPS POST to the Gemini API (gemini-2.5-flash-lite model). The prompt instructs Gemini to generate a concise, personalised study environment report that identifies the most significant

environmental issue and provides one actionable suggestion. AI is used here rather than rule-based logic because it produces natural, varied, and contextually appropriate feedback rather than repetitive fixed messages. A 20-second cooldown between API calls prevents rate limiting. All network operations are serialised through gNetworkMutex to prevent simultaneous WiFi calls from corrupting the driver state.

8.2 Data Sent and Received by ESP32S2 Board – Gemini API

Data sent to Gemini API: Study duration, average temperature, average humidity, average light level, average sound level, sound trigger count per 30 seconds, and overall environment condition classification. A prompt is given to the Gemini API to craft a guided response back to the ESP32S2 Board.

Data received from Gemini API: A plain text natural language response of up to 300 tokens containing an environment quality assessment and one actionable suggestion, returned as a JSON object with the response in the "text" field.

8.3 Telegram Bot API

At the end of each study session, when the Gemini report is generated, the full report text is forwarded to the Telegram group chat via bot.sendMessage(), delivering the AI-generated study feedback directly to all group members in real time. The ESP32 takes the network mutex before sending to prevent simultaneous WiFi operations from the Gemini and Telegram tasks from interfering with each other.

8.4 Data Sent and Received by ESP32S2 – Telegram Bot API

Data sent to Telegram Bot API: The plain text Gemini response string is sent to the Telegram Bot API via an HTTPS POST to api.telegram.org, authenticated using the bot token and addressed to the group using the hardcoded chat ID.

Data received from Telegram Bot API: A JSON response indicating whether the message was delivered successfully. The ESP32 does not process this response beyond checking for delivery confirmation in the Serial log

9. Additional Features

9.1: Exponential Moving Average (EMA)

Upon completion of a study session, the averaged light, sound, temperature, and humidity readings are transmitted to the Gemini API for environment analysis. Rather than using a Simple Moving Average (SMA), which requires storing and summing all past samples, our team implemented an Exponential Moving Average (EMA) to compute these running averages efficiently on the MCXC444.

The EMA update formula is:

$$avg = (1 - \alpha) \cdot avg + \alpha \cdot new = avg - \alpha \cdot avg + \alpha \cdot new$$

where α is the smoothing factor controlling how much weight is given to each new reading. Since we set $\alpha = 1/16$, the division can be replaced with a single bitwise right-shift by 4 (i.e., `FILTER_SHIFT = 4`), keeping the computation entirely in integer arithmetic with no floating-point overhead:

```
gAverageSensorData.light = gAverageSensorData.light - (gAverageSensorData.light >> FILTER_SHIFT)
                           + (light >> FILTER_SHIFT);
```

This means each new sample contributes only 6.25% to the running average, with older readings decaying in influence exponentially over time.

9.2: One-Shot Prompting

An additional feature implemented beyond the core requirements is a structured one-shot prompting pipeline through the Google Gemini API (gemini-2.5-flash-lite) that generates personalised study session feedback triggered automatically when a session ends. The prompt is carefully engineered in four layers: a system role injected via setChatSystemRole() that establishes Baymax from Big Hero 6 as the AI's persistent persona at the API level; a one-shot example input-output pair embedded directly in the prompt so the model can pattern-match the exact required structure rather than interpret an abstract description and generate a structure of its own; live sensor data substituted at runtime including session duration, average temperature, humidity, light sensor readings, and sound sensor readings count from the 30-second trigger window; and an explicit rules block enforcing output constraints such as Baymax tone throughout the message. Moreover, the model is configured with a maximum of 300 tokens (~200-250 words) to keep responses suitable for both the 128×64 OLED scroll display and Telegram delivery, with a temperature of 0.7 for naturally varied yet structurally reliable output.

10. Challenges faced & Solutions

Gemini API Connection Failure

The ESP32_AI_Connect library failed with a "connection refused" error when attempting to reach the Gemini API endpoint, despite the ESP32 being connected to WiFi. The root cause was that the library does not expose a setInsecure() method to bypass SSL certificate verification, which is required on the ESP32 as it lacks a certificate store. The solution was to replace the library entirely with a direct HTTPClient and WiFiClientSecureimplementation using setInsecure(), which resolved the connection issue.

Stack Overflow in Gemini task

vGeminiTask initially crashed with a stack overflow caused by the large amount of stack memory consumed when constructing the prompt String inside the task. The original prompt contained extensive numbered rules, ideal range comparisons, and detailed instructions, making the string concatenation very stack intensive. The issue was resolved by restructuring the prompt into a concise example-based format (using one-shot prompting), significantly reducing the stack memory required. The response from Gemini is handled by the ESP32_AI_Connect library on the heap rather than the task stack, so it was not the source of the overflow.

11. Conclusion

The Baymax successfully demonstrates an end-to-end IoT system built on FreeRTOS, combining real-time sensor monitoring, bidirectional UART communication, cloud-based AI inferencing, and instant Telegram messaging. The MCXC444 handles time-critical sensing and session management, while the ESP32 manages connectivity, display, and AI integration. The system meets all assignment requirements including multiple tasks, ISRs, mutexes, queues, bidirectional UART, and non-trivial cloud API usage. Through this project the team gained practical experience in real-time system design, embedded communications protocol development, FreeRTOS task coordination, and integrating consumer cloud APIs with constrained embedded hardware.